



C#.NET

Mr. Bangar Raju

 **NARESHO**
technologies

Inheritance (Part-I)

Understanding Inheritance in C#.NET: A Comprehensive Guide

By Albert Harmon

Published August 20, 2016

🕒 4 min read

Education



TLDR This blog post explores the concept of inheritance in C#.NET, explaining its significance, how to implement it, and the rules governing its use. Inheritance allows for code reusability and establishes a parent-child relationship between classes,

enabling derived classes to access members of base classes while adhering to specific access rules.

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows one class to inherit the properties and methods of another class. In this blog post, we will delve into the mechanics of inheritance in C#.NET, its advantages, and the rules that govern its implementation.

What is Inheritance?

Inheritance is a mechanism that enables a new class (known as the child or derived class) to inherit members (properties and methods) from an existing class (known as the parent or base class). This relationship is often described as a parent-child relationship. Just as children inherit property from their parents, a derived class can access the members of its base class.

The Importance of Inheritance

The primary advantage of inheritance is **code reusability**. Instead of copying and pasting code from one class to another, which can lead to redundancy and increased application size, inheritance allows developers to establish a relationship between classes. This means that a derived class can utilize the members of its parent class without duplicating code, thus improving application performance and maintainability.

Implementing Inheritance in C#.NET

To implement inheritance in C#.NET, you need to define a class that inherits from another class using the following syntax:

```
class ChildClass : ParentClass
```

In this syntax, `ChildClass`` is the derived class that inherits from `ParentClass``. This establishes a relationship where `ChildClass`` can access the members of `ParentClass``.

Example of Inheritance

Let's consider a practical example to illustrate inheritance. Suppose we have a base class named `ClassA`` with some methods:

```
public class ClassA
{
    public void MethodOne()
    {
        Console.WriteLine("Method One from Class A");
    }

    public void MethodTwo()
    {
        Console.WriteLine("Method Two from Class A");
    }
}
```

```
}  
}
```

Now, we can create a derived class ``classB`` that inherits from ``ClassA`` :

```
public class ClassB : ClassA  
{  
    public void MethodThree()  
    {  
        Console.WriteLine("Method Three from Class B");  
    }  
}
```

In this example, ``classB`` can access ``MethodOne`` and ``MethodTwo`` from ``ClassA`` as if they were its own methods. This demonstrates the reusability of code through inheritance.

Key Rules of Inheritance

While inheritance provides many benefits, there are important rules to keep in mind:

1. Accessibility of Parent Class Constructor

The constructor of the parent class must be accessible to the child class. If the constructor is private, the child class will not be able to inherit from the parent class. This is crucial because when an instance of the child class is created, it implicitly calls the constructor of the parent class to initialize its members.

2. Accessing Members

A child class can access the public and protected members of its parent class, but the parent class cannot access members that are defined solely in the child class. This reflects the legal principle that children have rights to their parents' property, but parents do not have rights to their children's property.

3. Initializing Parent Class Variables

A parent class variable can be initialized using a child class instance. This means that you can create a reference of the parent class type that points to an instance of the child class. However, while you can access the parent class members through this reference, you cannot access the child class members.

Practical Implementation

To see these principles in action, consider the following code snippet:

```
ClassB b = new ClassB();  
  
b.MethodOne(); // Inherited from ClassA  
b.MethodTwo(); // Inherited from ClassA  
b.MethodThree(); // Defined in ClassB
```

In this example, `b` is an instance of `ClassB`, which can call methods from both `ClassA` and its own methods. This illustrates how inheritance allows for the sharing of functionality between classes.

Conclusion

Inheritance is a powerful feature in C#.NET that promotes code reusability and establishes a clear hierarchy between classes. By understanding how to implement inheritance and the rules that govern it, developers can create more efficient and maintainable code. As you continue to explore object-oriented programming, keep these principles in mind to leverage the full potential of inheritance in your applications.